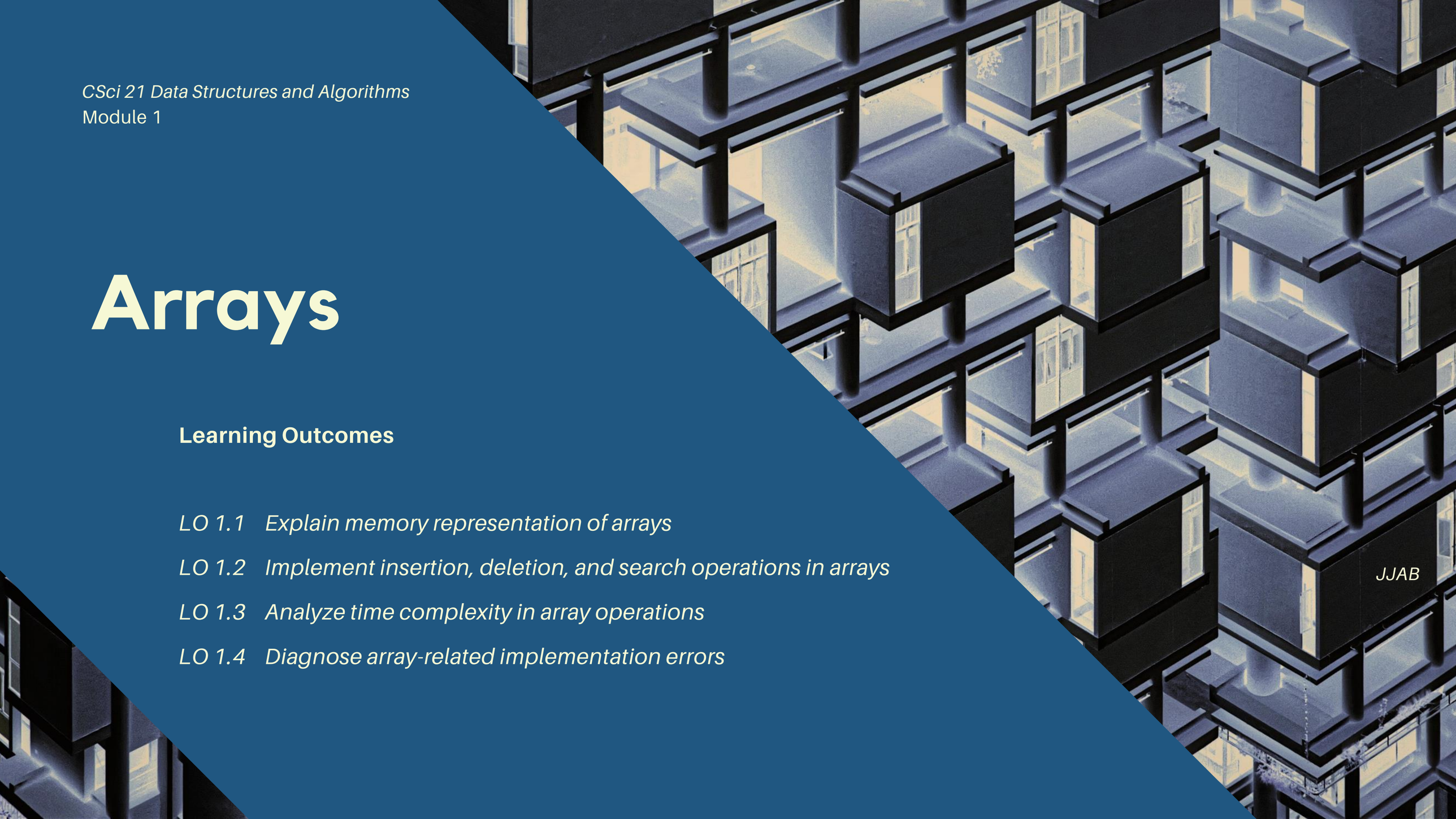
CSci 21 Data Structures and Algorithms
Module 1

Arrays



Learning Outcomes

LO 1.1 Explain memory representation of arrays

LO 1.2 Implement insertion, deletion, and search operations in arrays

LO 1.3 Analyze time complexity in array operations

LO 1.4 Diagnose array-related implementation errors

JJAB

Setting the Expectations

PREREQUISITES

- **Understood** the **basic programming concepts**—variable declaration, mathematical operations, selection and looping constructs, and functions.

- Utilized **arrays** in **any programming language**

What's an Array?

- Each hotel room has a number (*index*)
- Hotel rooms are side-by-side (*contiguous*)
- You are in room 202? Go to room 205 by walking 3 rooms after.

What's an Array?

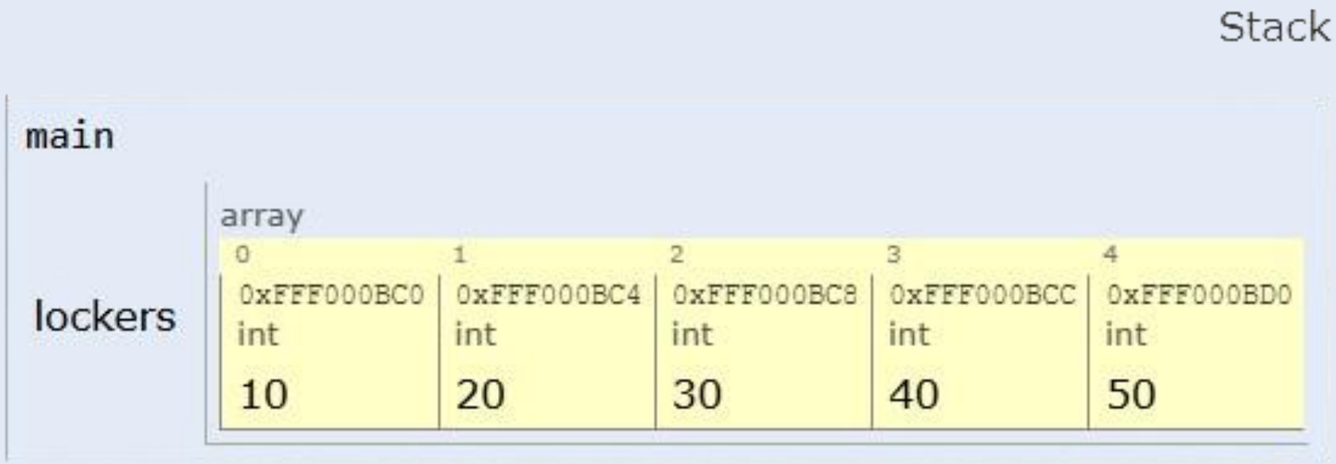
in the case of C++?

same-type

Contiguous blocks of memory storing items.

```
1  int lockers[5] = {10, 20, 30, 40, 50}; // 5 adjacent integers
```

Memory Representation



Base adress + (index ∗ size type) = Location of **arr[index]**

Example: lockers[2] = 0xFFF000BC0 + (2 × 4 bytes) = 0xFFF000BC8 → 30

Memory Representation

Why do you **CARE**?

💡 Access is O(1): CPU calculates address instantly

Searching an Element

```c
int linearSearch(int arr[], int size, int key) {
  for (int i = 0; i < size; i++) {  // Start at 0!
    if (arr[i] == key) return i;    // Found? Return index.
  }
  return -1; // Not found
}
```

Searching an Element

```c
int linearSearch(int arr[], int size, int key) {
  for (int i = 0; i < size; i++) {  // Start at 0!
    if (arr[i] == key) return i;    // Found? Return index.
  }
  return -1; // Not found
}
```

- Array is not sorted
- O(n)

Searching an Element

```c
int binarySearch(int arr[], int size, int key) {
  int left = 0, right = size - 1;  // right = size-1 (not size)!
  while (left <= right) {
    int mid = left + (right - left) / 2;  // Avoid overflow!
    if (arr[mid] == key) return mid;
    else if (arr[mid] < key) left = mid + 1;
    else right = mid - 1;
  }
  return -1;
}
```

Searching an Element

```c
int binarySearch(int arr[], int size, int key) {
  int left = 0, right = size - 1;  // right = size-1 (not size)!
  while (left <= right) {
    int mid = left + (right - left) / 2;  // Avoid overflow!
    if (arr[mid] == key) return mid;
    else if (arr[mid] < key) left = mid + 1;
    else right = mid - 1;
  }
  return -1;
}
```

- Array must be sorted
- O(log n)
- logarithm in base 2, why?
  - the search space is **halved**

Inserting an Element

```
1  void insert(int arr[], int& size, int cap, int k, int value) {
2    if (size >= cap) { cout << "Array Full!"; return; }
3    for (int i = size; i > k; i--) {  // Start from the end!
4      arr[i] = arr[i - 1];            // Shift right
5    }
6    arr[k] = value;
7    size++;
8  }
```

Inserting an Element

```
1  void insert(int arr[], int& size, int cap, int k, int value) {
2    if (size >= cap) { cout << "Array Full!"; return; }
3    for (int i = size; i > k; i--) {  // Start from the end!
4      arr[i] = arr[i - 1];            // Shift right
5    }
6    arr[k] = value;
7    size++;
8  }
```

- Insert at position k
- O(n)

Inserting an Element

```
1  void insertAtEnd(int arr[], int& size, int cap, int value) {
2    if (size >= cap) { cout << "Array Full!"; return; }
3    arr[size++] = value;
4  }
```

Inserting an Element

```
1  void insertAtEnd(int arr[], int& size, int cap, int value) {
2    if (size >= cap) { cout << "Array Full!"; return; }
3    arr[size++] = value;
4  }
```

- Insert at the end
- O(1)

Deleting an Element

```
1  void deleteAt(int arr[], int& size, int k) {
2    for (int i = k; i < size - 1; i++) {  // Stop at size-2!
3      arr[i] = arr[i + 1];                // Shift left
4    }
5    size--;
6  }
```

Deleting an Element

```
1  void deleteAt(int arr[], int& size, int k) {
2    for (int i = k; i < size - 1; i++) {  // Stop at size-2!
3      arr[i] = arr[i + 1];                // Shift left
4    }
5    size--;
6  }
```

- Delete element at index k
- O(n)

Complexity of Array Operations

OPERATION	TIME COMPLEXITY	WHY?
Access at position k	O(1)	Pointer arithmetic
Linear search	O(n)	Search all elements; worst case found at the last
Binary search	O(log n)	Halve search space in each iteration
Insert at position k	O(n)	Worst case insertion at index 0 shift all elements to the right
Insert at rear	O(1)	No shifting of elements; location is pointer arithmetic
Delete at position k	O(n)	Worst case deletion at index 0 shift all elements to the left

Debugging Array Errors

1. Off-by-one

```
1  int search(int arr[], int size, int key) {
2    for (int i = 0; i <= size; i++) {
3      if (arr[i] == key) return i;
4    }
5    return -1;
6  }
```

Debugging Array Errors

1. Off-by-one

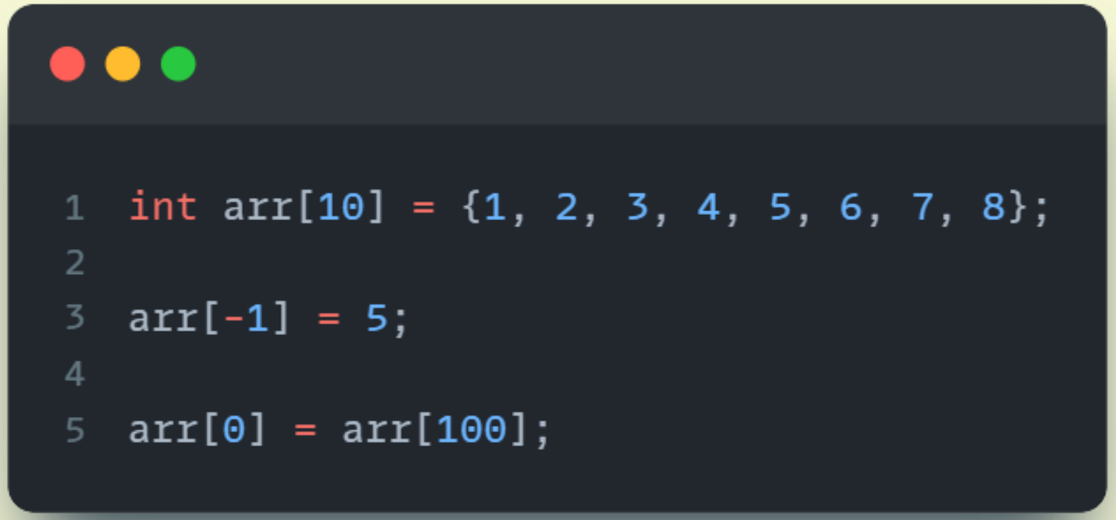
```
1  int search(int arr[], int size, int key) {
2    for (int i = 0; i <= size; i++) {
3      if (arr[i] == key) return i;
4    }
5    return -1;
6  }
```

❌ Line 2 include `i = size`
Line 3 accesses `arr[size]`

✅ `i < size` (valid indices: 0 to size-1)

Debugging Array Errors

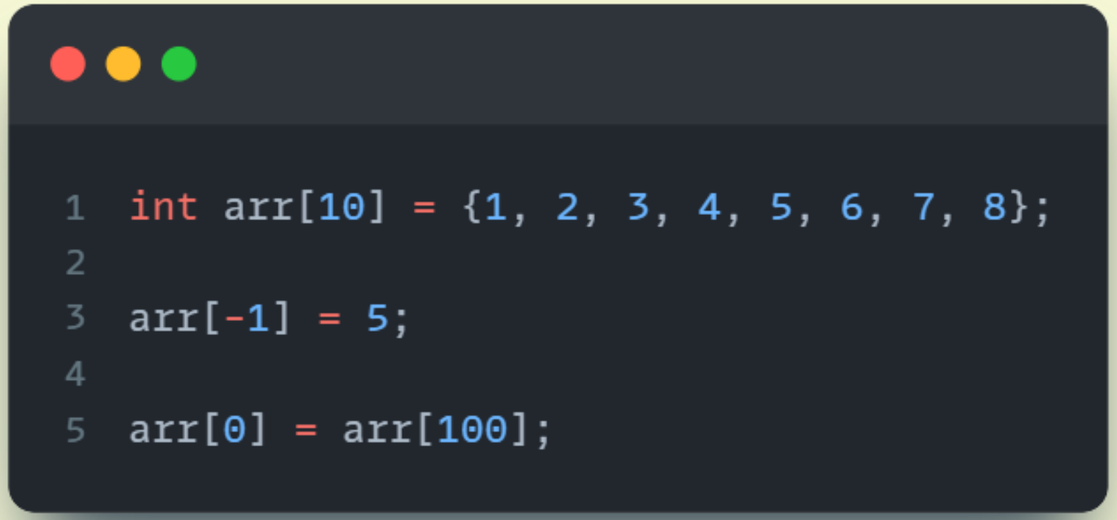
2. Out-of-bounds access / Buffer overflow or underflow



```
1  int arr[10] = {1, 2, 3, 4, 5, 6, 7, 8};
2
3  arr[-1] = 5;
4
5  arr[0] = arr[100];
```

Debugging Array Errors

2. Out-of-bounds access / Buffer overflow or underflow



```
1  int arr[10] = {1, 2, 3, 4, 5, 6, 7, 8};
2
3  arr[-1] = 5;
4
5  arr[0] = arr[100];
```

❌ Line 3 accesses a negative index
Line 5 accesses > index 9 (size - 1)

✅ Only access index between 0 and size - 1

Debugging Array Errors

3. *Forgetting to update size*

Always update size after insertion/deletion

4. *Shift direction errors*

Insertion: shift **right** (start from *end*)
Deletion: shift **left** (start from *position*)

Applications

Arrays are **EVERYWHERE!**

Caches, matrices in image processing, databases, OOP collections, etc.

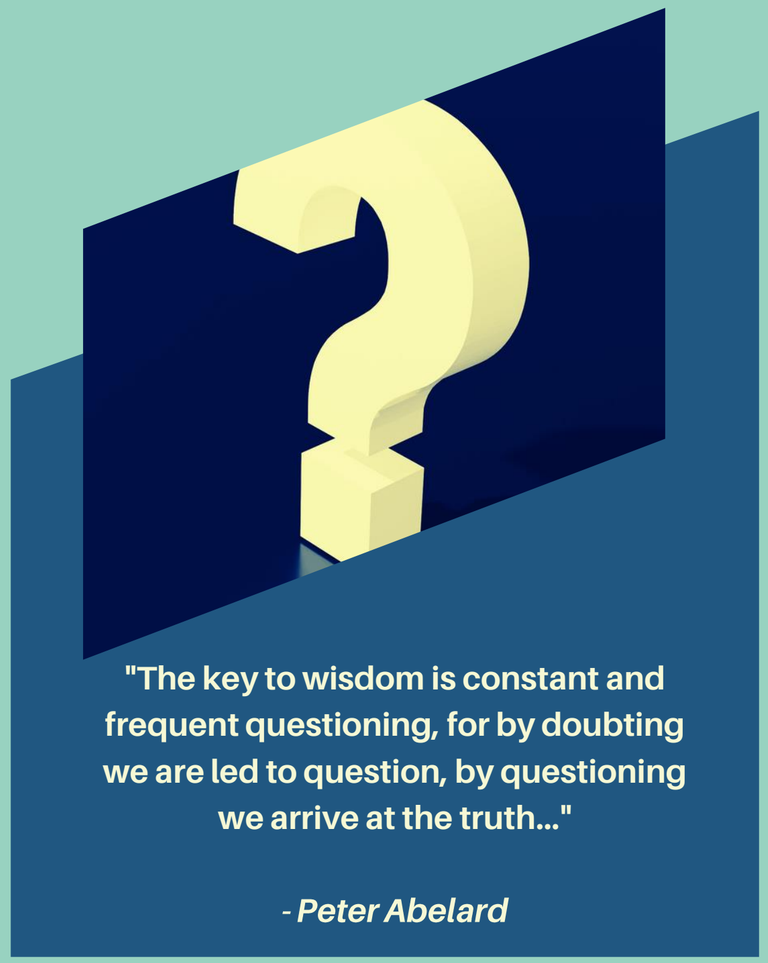
Limitations

Static/stack arrays → fixed size

Dynamic arrays (vectors) fixes this but **huge memory footprint**

Golden Rule

“Contiguous memory = lighting-fast access.
Shifting elements = costly.
Choose wisely.”



OPEN FORUM Q/A

for Module 1